Кременчуцький національний університет імені Михайла Остроградського
(повне найменування вищого навчального закладу)

Кафедра комп'ютерної інженерії та електроніки
(повна назва кафедри)

ЛАБОРАТОРНА РОБОТА № 4

Тема: Обробка одновимірних і багатовимірних списків та операції з кортежами

Мета роботи: Опанувати практичні навички проектування та реалізації алгоритмів обробки одновимірних і багатовимірних структур даних мовою Python. Навчитися здійснювати маніпуляції з динамічними списками (list), застосовувати адресну індексацію, операції вилучення зрізів (slicing) та декларативні генератори списків (List Comprehensions). Дослідити алгоритмічні засади сортування даних прямими методами та стандартним гібридним алгоритмом Timsort. Опанувати математичний апарат матричної алгебри (транспонування, множення матриць, пошук сідлових точок) на базі вкладених списків, а також вивчити концепцію збереження незмінного стану конфігурацій за допомогою кортежів (tuple).

Студента III курсу КІ-24-1 групи
Ступінь вищої освіти бакалавр
(бакалавр, магістр)
Спеціальність 123 – «Комп'ютерна інженерія»
Освітня програма Комп'ютерна інженерія
Яцентюка Євгенія Олександровича

м. Кременчук 2026 рік

1. Мета роботи та математична постановка задачі

Мета роботи: опанувати практичні навички проєктування та реалізації алгоритмів обробки одновимірних і багатовимірних структур даних мовою Python; навчитися працювати зі списками (list), генераторами списків (List Comprehensions) і кортежами (tuple); реалізувати пряме сортування вручну та порівняти його зі стандартним алгоритмом Timsort; виконати базові операції матричної алгебри на вкладених списках.

1.1. Індивідуальне завдання (варіант 2)

Частина	**Умова**
Вектор V	N = 20, діапазон [0, 100], вилучення елементів, кратних 5; сортування: Bubble Sort
Матриця A	розмір 3 × 5; множення на вектор-стовпець $X \in R^5$
Результат	вектор Y = A × X, мінімум кожного рядка матриці, пошук сідлових точок
Кортеж	CONFIG = (N, M, K, X_MIN, X_MAX) = (20, 3, 5, 0, 100) з паралельним розпакуванням

1.2. Математичний опис

Вектор $V = (v_0, v_1, \ldots, v_{19})$, де v_i — випадкові цілі числа з діапазону [0, 100]. Результатом фільтрації є вектор V', що містить лише ті елементи, які не діляться націло на 5:

$$V' = \{ v_i \in V : v_i \bmod 5 \neq 0 \}$$

Число 0 кратне 5, тому якщо воно потрапить до вектора, його також буде вилучено. Далі вектор V' упорядковується за зростанням: $v_j' \leq v_{j+1}'$ для всіх j.

Для матриці $A \in Z^{3x5}$ та вектора-стовпця $X \in Z^{5x1}$ добуток Y = A × X визначається формулою:

$$Y_i = \Sigma_{k=0}^{4} A_{ik} \cdot X_k, \quad i = 0, 1, 2$$

Мінімум кожного рядка матриці:

$$m_i = min\ A_{ij}, \quad 0 \le j < 5$$

Сідловою точкою матриці називається елемент (i*, j*), який одночасно є мінімальним у своєму рядку та максимальним у своєму стовпці:

$$A(i^*, j^*) = min\ A_i{}^*{}_j = max\ A_{ij}{}^*$$

2. Алгоритмічний аналіз

2.1. Бульбашкове сортування (Bubble Sort)

Алгоритм виконує послідовні проходи по масиву, порівнюючи сусідні елементи та міняючи їх місцями, якщо вони стоять у хибному порядку. Після i-го проходу найбільший із необроблених елементів «спливає» на позицію n − 1 − i, тому внутрішній цикл на кожному наступному проході коротшає на один елемент.

Формальний опис алгоритму:

1) створити копію arr вхідного списку, n = len(arr), лічильники порівнянь і обмінів дорівнюють 0;
2) для i від 0 до n − 2: встановити прапорець swapped = False;
3) для j від 0 до n − 2 − i: порівняти arr[j] і arr[j + 1]; якщо arr[j] > arr[j + 1], обміняти їх місцями та встановити swapped = True;
4) якщо після внутрішнього циклу swapped = False — масив уже впорядкований, вийти з циклу;
5) повернути arr, кількість порівнянь і кількість обмінів.

Обмін виконується через кортежне розпакування arr[j], arr[j + 1] = arr[j + 1], arr[j], що не потребує тимчасової змінної.

Випадок	**Часова складність**	**Примітка**
Найкращий	O(N)	масив уже впорядкований, один прохід без обмінів
Середній	$O(N^2)$	близько $N^2/4$ обмінів
Найгірший	$O(N^2)$	зворотний порядок, N(N − 1)/2 обмінів

Випадок	Часова складність	Примітка
Додаткова пам’ять	O(1)	сортування на місці (без урахування копії вхідного списку)

Алгоритм стійкий: рівні елементи не змінюють взаємного порядку, оскільки обмін виконується лише за строгої нерівності arr[j] > arr[j + 1].

2.2. Множення матриці на вектор-стовпець

Вектор-стовпець X подається як матриця розмірності 5 × 1 (список із п’яти одноелементних списків). Це дає змогу використати одну універсальну функцію multiply_matrices для будь-яких сумісних матриць. Перед множенням перевіряється умова len(A[0]) == len(B); якщо вона порушена, генерується ValueError. Кожен елемент результату обчислюється як скалярний добуток рядка A на стовпець B. Складність для A (m × p) та B (p × n) становить O(m · n · p); для варіанта 2 це 3 · 1 · 5 = 15 операцій множення.

2.3. Пошук мінімуму рядків і сідлових точок

Спочатку за один прохід обчислюються мінімуми всіх рядків (масив row_minima) та максимуми всіх стовпців (масив col_maxima). Потім кожен елемент перевіряється на умову A[i][j] == row_minima[i] and A[i][j] == col_maxima[j]. Таке розбиття дозволяє уникнути повторного обчислення мінімумів і максимумів для кожного елемента: загальна складність становить O(M · K), тоді як наївна перевірка кожного елемента давала б O(M · K · (M + K)).

2.4. Використання кортежів

Параметри генерації передаються у вигляді незмінного кортежу CONFIG = (N, M, K, X_MIN, X_MAX) = (20, 3, 5, 0, 100). У функції main він розпаковується одним оператором n, m, k, x_min, x_max = CONFIG. Незмінність кортежу гарантує, що параметри не буде випадково змінено під час виконання. Кортежі також використовуються як тип повернення функцій

(find_extremes, bubble_sort_manual, find_saddle_points) і для обміну елементів під час сортування.

3. Програмний код

3.1. Файл vector_algorithms.py

```
"""
Лабораторна робота № 4. Модуль 1. Варіант 2.
Обробка одновимірного вектора: генерація, вилучення елементів, кратних 5,
пошук екстремумів та ручна реалізація бульбашкового сортування (Bubble Sort).
Конфігурація передається у вигляді незмінного кортежу.
Освітній компонент: Програмування на Python.
"""

import random
import time

# Незмінний конфігураційний кортеж параметрів (варіант 2):
# CONFIG = (N, M, K, X_MIN, X_MAX)
#   N      - довжина вектора V
#   M, K   - розмірність матриці A (використовуються в matrix_operations.py)
#   X_MIN, X_MAX - межі генерації випадкових цілих чисел
CONFIG: tuple[int, int, int, int, int] = (20, 3, 5, 0, 100)


def generate_random_vector(length: int, min_val: int, max_val: int) -> list[int]:
    """Генерує вектор випадкових цілих чисел довжини length у діапазоні [min_val,
max_val]."""
    if length <= 0:
        raise ValueError("Довжина вектора повинна бути строго додатною.")
    if min_val > max_val:
        raise ValueError("Нижня межа діапазону не може перевищувати верхню.")

    return [random.randint(min_val, max_val) for _ in range(length)]


def remove_multiples_of(vector: list[int], divisor: int) -> list[int]:
    """
    Вилучає з вектора усі елементи, кратні divisor (List Comprehension).
    Нуль кратний будь-якому числу, тому також вилучається.
    """
    if divisor == 0:
        raise ValueError("Дільник не може дорівнювати нулю.")

    return [x for x in vector if x % divisor != 0]


def find_extremes(vector: list[int]) -> tuple[int, int, int, int]:
```

```
    """
    Знаходить мінімум і максимум вектора разом з індексами першого входження.
    Повертає кортеж (min_value, min_index, max_value, max_index).
    Реалізовано одним проходом за O(N) без вбудованих min()/max().
    """
    if not vector:
        raise ValueError("Неможливо знайти екстремуми порожнього вектора.")

    min_idx = max_idx = 0
    for i in range(1, len(vector)):
        if vector[i] < vector[min_idx]:
            min_idx = i
        if vector[i] > vector[max_idx]:
            max_idx = i

    return vector[min_idx], min_idx, vector[max_idx], max_idx


def bubble_sort_manual(input_list: list[int]) -> tuple[list[int], int, int]:
    """
    Бульбашкове сортування (Bubble Sort) за зростанням, без .sort() і sorted().
    Працює з копією, тому вхідний список не змінюється.
    Містить оптимізацію раннього виходу: якщо за прохід не було жодного обміну,
    масив уже впорядкований.
    Повертає кортеж (відсортований_список, кількість_порівнянь,
кількість_обмінів).
    Часова складність: O(N^2) у середньому та найгіршому, O(N) у найкращому
випадку.
    """
    arr = list(input_list)
    n = len(arr)
    comparisons = 0
    swaps = 0

    for i in range(n - 1):
        swapped = False
        # Після i проходів останні i елементів уже стоять на своїх місцях
        for j in range(n - 1 - i):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                # Паралельний обмін через кортежне розпакування
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swaps += 1
                swapped = True
        if not swapped:
            break

    return arr, comparisons, swaps


def main() -> None:
```

```
    """Головна функція демонстрації обробки вектора (варіант 2)."""
    # Паралельне розпакування конфігураційного кортежу
    n, m, k, x_min, x_max = CONFIG

    # Фіксоване зерно для повторюваності результатів
    random.seed(42)
    raw_vector = generate_random_vector(n, x_min, x_max)

    print("=" * 80)
    print(f"{'ВАРІАНТ 2. ОБРОБКА ВЕКТОРА ТА БУЛЬБАШКОВЕ СОРТУВАННЯ':^80}")
    print("=" * 80)
    print(f"CONFIG (розпакований tuple): N = {n}, M = {m}, K = {k}, "
          f"діапазон = [{x_min}, {x_max}]")
    print(f"Початковий вектор V ({len(raw_vector)} ел.):")
    print(f"  {raw_vector}")

    # 1. Вилучення елементів, кратних 5
    filtered = remove_multiples_of(raw_vector, 5)
    removed = [x for x in raw_vector if x % 5 == 0]
    print(f"\nВилучені елементи, кратні 5 ({len(removed)} шт.): {removed}")
    print(f"Вектор після вилучення ({len(filtered)} ел.):")
    print(f"  {filtered}")

    # 2. Екстремуми відфільтрованого вектора
    if filtered:
        v_min, i_min, v_max, i_max = find_extremes(filtered)
        print(f"\nМінімум: {v_min} (індекс {i_min}), максимум: {v_max} (індекс
{i_max})")
    else:
        print("\nПісля фільтрації вектор порожній, екстремумів немає.")

    # 3. Бульбашкове сортування (ручна реалізація)
    t_start = time.perf_counter()
    sorted_manual, cmp_count, swap_count = bubble_sort_manual(filtered)
    t_bubble = time.perf_counter() - t_start
    print(f"\nBubble Sort за зростанням:")
    print(f"  {sorted_manual}")
    print(f"Порівнянь: {cmp_count}, обмінів: {swap_count}, час: {t_bubble *
1000:.4f} мс")

    # 4. Верифікація зі стандартним Timsort
    t_start = time.perf_counter()
    sorted_builtin = sorted(filtered)
    t_timsort = time.perf_counter() - t_start
    print(f"\nTimsort (sorted): час {t_timsort * 1000:.4f} мс")
    print(f"Результати співпадають: {sorted_manual == sorted_builtin}")

    # 5. Перевірка, що вихідний вектор не було змінено
    print(f"Вихідний вектор V не змінено: {len(raw_vector) == n}")
    print("=" * 80)
```

```
if __name__ == "__main__":
    t_total = time.perf_counter()
    main()
    elapsed = time.perf_counter() - t_total
    print(f"Час виконання модуля векторних алгоритмів: {elapsed * 1000:.4f}
мс\n")
```

3.2. Файл matrix_operations.py

```
"""
Лабораторна робота № 4. Модуль 2. Варіант 2.
Матрична алгебра: генерація матриці A (3 x 5), множення на вектор-стовпець X,
пошук мінімуму кожного рядка та пошук сідлових точок.
Освітній компонент: Програмування на Python.
"""

import random
import time

# Незмінний конфігураційний кортеж параметрів (варіант 2):
# CONFIG = (N, M, K, X_MIN, X_MAX)
CONFIG: tuple[int, int, int, int, int] = (20, 3, 5, 0, 100)


def generate_matrix(rows: int, cols: int, min_val: int, max_val: int) ->
list[list[int]]:
    """Генерує матрицю rows x cols вкладеними генераторами списків."""
    if rows <= 0 or cols <= 0:
        raise ValueError("Кількість рядків і стовпців повинна бути строго
додатною.")
    if min_val > max_val:
        raise ValueError("Нижня межа діапазону не може перевищувати верхню.")

    # Кожен рядок створюється окремо, тому спільного стану між рядками немає
    return [[random.randint(min_val, max_val) for _ in range(cols)] for _ in
range(rows)]


def generate_column_vector(length: int, min_val: int, max_val: int) ->
list[list[int]]:
    """Генерує вектор-стовпець X розмірності length x 1 у вигляді списку
одноелементних рядків."""
    return generate_matrix(length, 1, min_val, max_val)


def multiply_matrices(matrix_a: list[list[int]], matrix_b: list[list[int]]) ->
list[list[int]]:
    """
```

```
    Обчислює матричний добуток C = A x B за формулою C[i][j] = sum(A[i][k] * B[k]
[j]).
    Кількість стовпців A повинна дорівнювати кількості рядків B, інакше -
ValueError.
    Складність: O(m * n * p).
    """
    rows_a, cols_a = len(matrix_a), len(matrix_a[0])
    rows_b, cols_b = len(matrix_b), len(matrix_b[0])

    if cols_a != rows_b:
        raise ValueError(
            f"Несумісні розмірності для множення: A({rows_a}x{cols_a}) та
B({rows_b}x{cols_b})."
        )

    return [
        [sum(matrix_a[i][k] * matrix_b[k][j] for k in range(cols_a)) for j in
range(cols_b)]
        for i in range(rows_a)
    ]


def row_minima(matrix: list[list[int]]) -> list[tuple[int, int, int]]:
    """
    Знаходить мінімум кожного рядка матриці.
    Повертає список кортежів (індекс_рядка, індекс_стовпця_мінімуму, значення).
    """
    result = []
    for i, row in enumerate(matrix):
        min_j = 0
        for j in range(1, len(row)):
            if row[j] < row[min_j]:
                min_j = j
        result.append((i, min_j, row[min_j]))
    return result


def find_saddle_points(matrix: list[list[int]]) -> list[tuple[int, int, int]]:
    """
    Знаходить усі сідлові точки: елементи, що є мінімальними у своєму рядку
    та одночасно максимальними у своєму стовпці.
    Повертає список кортежів (індекс_рядка, індекс_стовпця, значення).
    """
    rows, cols = len(matrix), len(matrix[0])

    minima = [min(row) for row in matrix]
    maxima = [max(matrix[r][c] for r in range(rows)) for c in range(cols)]

    return [
        (i, j, matrix[i][j])
        for i in range(rows)
```

```
        for j in range(cols)
        if matrix[i][j] == minima[i] and matrix[i][j] == maxima[j]
    ]


def print_matrix(matrix: list[list[int]], title: str) -> None:
    """Виводить матрицю у вигляді вирівняної таблиці."""
    print(f"\n--- {title} ({len(matrix)}x{len(matrix[0])}) ---")
    for row in matrix:
        print("  [ " + " ".join(f"{elem:6d}" for elem in row) + " ]")


def print_saddles(saddles: list[tuple[int, int, int]]) -> None:
    """Виводить результат пошуку сідлових точок."""
    if not saddles:
        print("  Сідлових точок не виявлено.")
        return
    for idx, (r, c, v) in enumerate(saddles, start=1):
        print(f"  Точка {idx}: рядок = {r}, стовпець = {c}, значення = {v}")


def main() -> None:
    """Головна функція демонстрації матричних операцій (варіант 2)."""
    n, m, k, x_min, x_max = CONFIG  # паралельне розпакування кортежу

    random.seed(101)

    print("=" * 80)
    print(f"{'ВАРІАНТ 2. МАТРИЦЯ 3x5, ДОБУТОК A x X ТА СІДЛОВІ ТОЧКИ':^80}")
    print("=" * 80)
    print(f"CONFIG: M = {m}, K = {k}, діапазон елементів = [{x_min}, {x_max}]")

    # 1. Матриця A (M x K) і вектор-стовпець X (K x 1)
    mat_a = generate_matrix(m, k, x_min, x_max)
    vec_x = generate_column_vector(k, x_min, x_max)
    print_matrix(mat_a, "Матриця A")
    print_matrix(vec_x, "Вектор-стовпець X")

    # 2. Y = A x X
    vec_y = multiply_matrices(mat_a, vec_x)
    print_matrix(vec_y, "Вектор Y = A x X")

    # Контрольна перевірка першого елемента вручну
    terms = " + ".join(f"{mat_a[0][t]}*{vec_x[t][0]}" for t in range(k))
    manual_y0 = sum(mat_a[0][t] * vec_x[t][0] for t in range(k))
    print(f"\nПеревірка Y[0] = {terms} = {manual_y0}  ->  збігається: {manual_y0
== vec_y[0][0]}")

    # 3. Мінімум кожного рядка матриці A
    print("\nМінімум кожного рядка матриці A:")
    for i, j, v in row_minima(mat_a):
```

```
        print(f"  Рядок {i}: min = {v} (стовпець {j})")

    # 4. Сідлові точки матриці A
    print("\nСідлові точки згенерованої матриці A:")
    print_saddles(find_saddle_points(mat_a))

    # 5. Контрольні матриці з відомим результатом
    control_1 = [
        [7, 8, 9],
        [4, 5, 6],
        [1, 2, 3],
    ]
    control_2 = [
        [5, 5, 6],
        [5, 5, 7],
        [1, 2, 8],
    ]
    print_matrix(control_1, "Контрольна матриця M1")
    print("Сідлові точки M1 (очікується одна: (0, 0) = 7):")
    print_saddles(find_saddle_points(control_1))

    print_matrix(control_2, "Контрольна матриця M2")
    print("Сідлові точки M2 (очікується чотири, усі зі значенням 5):")
    print_saddles(find_saddle_points(control_2))

    # 6. Перевірка контролю розмірностей
    print("\nПеревірка контролю розмірностей (A 3x5 на матрицю 4x1):")
    try:
        multiply_matrices(mat_a, [[1], [2], [3], [4]])
    except ValueError as err:
        print(f"  Перехоплено ValueError: {err}")

    print("=" * 80)


if __name__ == "__main__":
    t_start = time.perf_counter()
    main()
    elapsed = time.perf_counter() - t_start
    print(f"Час виконання модуля матричної алгебри: {elapsed * 1000:.4f} мс\n")
```

4. Результати обчислень

Запуск виконувався в середовищі Conda ce_lab_env командами python vector_algorithms.py та python matrix_operations.py.

4.1. Результат роботи vector_algorithms.py

```
PS C:\Users\Janis\Desktop\урок\Програмування на Python. Базовий рівень\lab4\lab04\lab04> & C:\Users\Janis\miniconda3\envs\
ce_lab_env\python.exe "c:/Users/Janis/Desktop/урок/Програмування на Python. Базовий рівень/lab4/lab04/lab04/vector_algorit
hms.py"
============================================================================
              ВАРІАНТ 2. ОБРОБКА ВЕКТОРА ТА БУЛЬБАШКОВЕ СОРТУВАННЯ
============================================================================
CONFIG (розпакований tuple): N = 20, M = 3, K = 5, діапазон = [0, 100]
Початковий вектор V (20 ел.):
  [81, 14, 3, 94, 35, 31, 28, 17, 94, 13, 86, 94, 69, 11, 75, 54, 4, 3, 11, 27]

Вилучені елементи, кратні 5 (2 шт.): [35, 75]
Вектор після вилучення (18 ел.):
  [81, 14, 3, 94, 31, 28, 17, 94, 13, 86, 94, 69, 11, 54, 4, 3, 11, 27]

Мінімум: 3 (індекс 2), максимум: 94 (індекс 3)

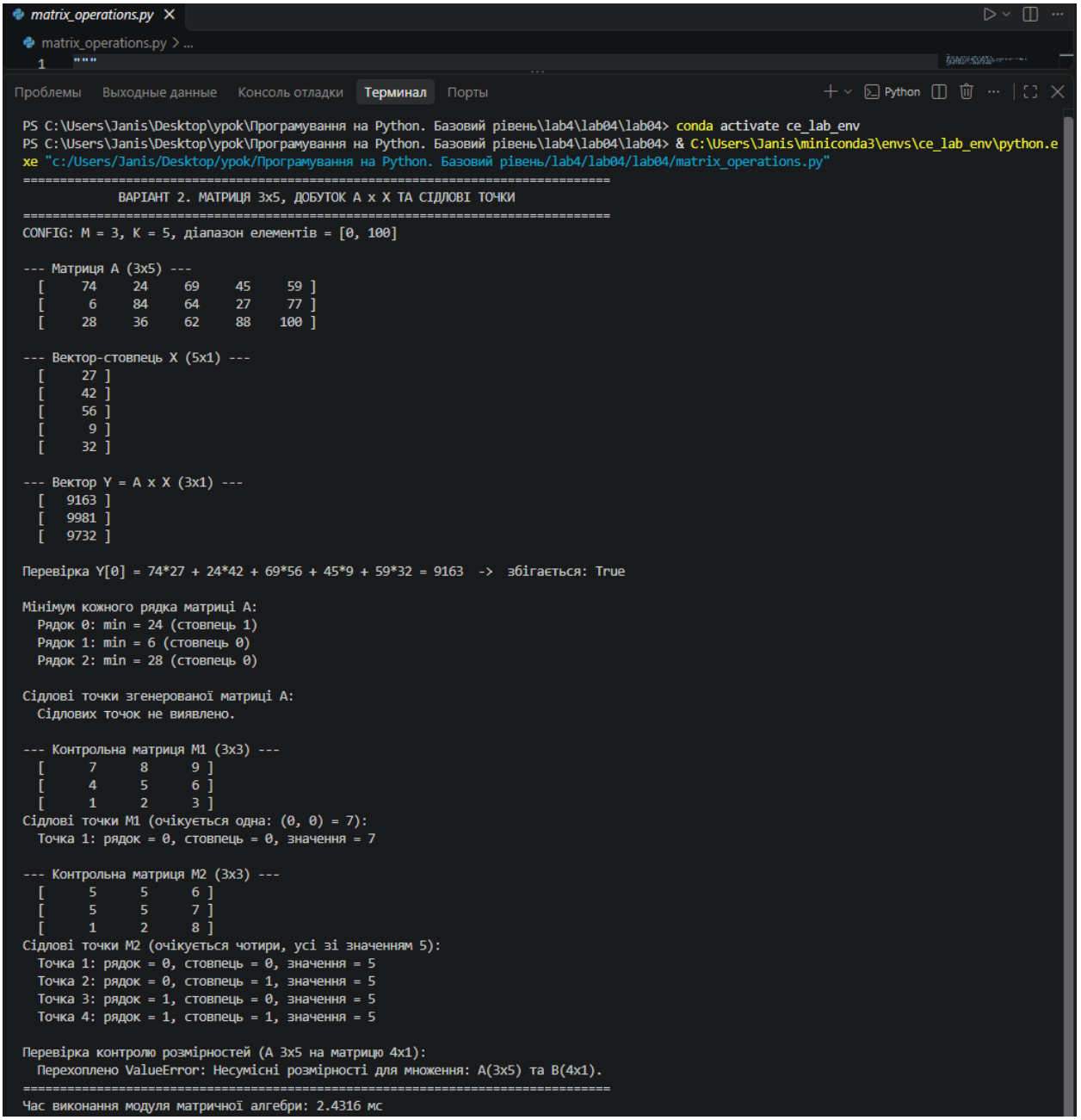
Bubble Sort за зростанням:
  [3, 3, 4, 11, 11, 13, 14, 17, 27, 28, 31, 54, 69, 81, 86, 94, 94, 94]
Порівнянь: 150, обмінів: 91, час: 0.1238 мс

Timsort (sorted): час 0.0034 мс
Результати співпадають: True
Вихідний вектор V не змінено: True
============================================================================
Час виконання модуля векторних алгоритмів: 1.3678 мс
```

Скріншот 1 — Консольний вивід модуля обробки вектора

З початкового вектора з 20 елементів вилучено два елементи, кратні 5 (35 та 75), тому в результаті залишилось 18 елементів. Мінімум становить 3 (індекс 2), максимум — 94 (індекс 3). Ручне бульбашкове сортування виконало 150 порівнянь і 91 обмін; результат збігається з вбудованим Timsort. Кількість порівнянь менша за теоретичну $18 \cdot 17 / 2 = 153$ завдяки оптимізації раннього виходу.

4.2. Результат роботи matrix_operations.py



Скріншот 2 — Консольний вивід модуля матричної алгебри

Для перевірки правильності множення вручну обчислено перший елемент: $Y_0 = 74 \cdot 27 + 24 \cdot 42 + 69 \cdot 56 + 45 \cdot 9 + 59 \cdot 32 = 1998 + 1008 + 3864 + 405 + 1888 = 9163$, що збігається з результатом програми. Мінімуми рядків матриці A: 24, 6 та 28. У згенерованій матриці сідлових точок немає, що для випадкових матриць із широким діапазоном значень є типовою ситуацією. Коректність алгоритму перевірено на двох контрольних матрицях: M1 має одну сідлову точку (0, 0) зі значенням 7, а M2 — чотири сідлові точки зі

значенням 5, що підтверджує коректну обробку рівних значень. Спроба помножити матрицю 3 × 5 на матрицю 4 × 1 призводить до очікуваного ValueError.

5. Посилання на віддалений репозиторій

Профіль: https://github.com/kefir4ikk

Репозиторій лабораторної роботи :
https://github.com/kefir4ikk/progPython_YevgeniiYatsentiuk/tree/master/labs/lab4

6. Висновки

У ході лабораторної роботи реалізовано обробку одновимірного вектора та матриці мовою Python для варіанта 2. Список у CPython зберігає масив покажчиків, тому доступ за індексом має складність O(1), а двовимірна матриця природно моделюється вкладеними списками з доступом A[i][j] за O(1).

Генератори списків дозволили стисло виразити генерацію вектора та матриці й вилучення елементів, кратних 5, без явного виклику append. Створення кожного рядка матриці окремим генератором виключає помилку спільного стану, яка виникає у виразі [[0] * 4] * 3.

Бульбашкове сортування, реалізоване вручну, має складність $O(N^2)$ у середньому та найгіршому випадках і O(N) у найкращому завдяки прапорцю раннього виходу. Для вектора з 18 елементів воно виконало 150 порівнянь і 91 обмін та дало той самий результат, що й вбудований Timsort зі складністю O(N log N). Для малих масивів різниця непомітна, але зі зростанням N кубічне та квадратичне зростання витрат робить прямі алгоритми непридатними для великих даних.

Множення матриці на вектор-стовпець із контролем розмірностей і пошук сідлових точок зі складністю O(M · K) перевірено на контрольних прикладах. Використання незмінного кортежу CONFIG для передавання параметрів та кортежів як значень, що повертаються функціями, робить код

надійнішим і зрозумілішим, оскільки захищає конфігурацію від випадкових змін.

Контрольні питання:

1. Як влаштована структура PyListObject? Чому L[i] має складність O(1), а L.insert(0, x) — O(n)?

PyListObject містить покажчик ob_item на неперервний масив покажчиків на об’єкти в купі та лічильники розміру й виділеної ємності. Адреса елемента обчислюється як ob_item + i · 8 байтів, тому доступ за індексом виконується за сталий час. Під час вставки на початок усі n наявних покажчиків потрібно зсунути на одну позицію вправо, що дає лінійну складність O(n).

2. Поясніть причину виникнення помилки спільного стану при створенні нульової матриці за допомогою виразу A = [[0] * 4] * 3. Яким чином використання генераторів списків усуває цю проблему?

Множення списку на число повторює не вміст, а посилання на той самий внутрішній список. У результаті всі три рядки — це один об’єкт, і зміна A[0][0] змінює елемент у кожному рядку. Генератор [[0] * 4 for _ in range(3)] на кожній ітерації створює новий окремий внутрішній список.

3. У чому полягає математичний зміст операції вилучення зрізу (slicing)? Чи створює операція sub = L[1:5] новий масив у пам'яті, і як це впливає на вихідний список?

Зріз повертає нову послідовність, що містить елементи з вказаного діапазону індексів [1, 5). Для списку створюється новий об’єкт list із копіями покажчиків (поверхнева копія), тому вихідний список не змінюється; самі вкладені об’єкти при цьому не копіюються.

4. Сформулюйте математичну умову існування сідлової точки в матриці. Яка максимальна та мінімальна кількість сідлових точок може існувати у довільній прямокутній матриці?

Елемент A[i][j] є сідловою точкою, якщо він мінімальний у своєму рядку та максимальний у своєму стовпці. Мінімальна кількість сідлових точок — 0 (як у згенерованій матриці A у цій роботі). Максимальна — M · K, коли всі елементи матриці однакові: тоді кожен елемент одночасно є і мінімумом рядка, і максимумом стовпця.

5. Опишіть принцип роботи алгоритму сортування вибором (Selection Sort). Якою є його часова складність у найкращому, середньому та найгіршому випадках?

На і-му кроці в невпорядкованій частині знаходиться мінімальний елемент і ставиться на позицію і. Кількість порівнянь не залежить від вхідних даних, тому складність дорівнює $O(n^2)$ у найкращому, середньому та найгіршому випадках; кількість обмінів не перевищує $n - 1$.

6. Чому об'єкти типу tuple є незмінними, і які архітектурні переваги це надає інтерпретатору CPython щодо оптимізації виділення оперативної пам'яті?

Кортеж має фіксований розмір, і в його інтерфейсі немає операцій зміни елементів. Це дозволяє не резервувати надлишкову пам’ять (over-allocation), повторно використовувати вільні блоки малих кортежів, обчислювати константні кортежі на етапі компіляції та використовувати кортеж, що складається з хешованих елементів, як ключ словника. Також незмінність захищає конфігураційні дані від випадкової модифікації.